

How to Present This as Work Experience in an Interview

Use this as a script. Practice saying it out loud. The interviewer doesn't care about class names — they care about your design thinking.

When They Ask: "Tell me about a project you worked on"

Opening (set the context):

"I worked on a full-stack web application for managing student records — things like enrollment, course assignments, and student profiles. It was built as a microservice architecture with three independently deployable services."

The three services:

*"We had a **React frontend** as the user-facing application, a **core backend API** built with Spring Boot that handled all the CRUD operations, and a separate **authentication microservice** that was responsible for user management and token-based security. Each service ran on its own port and could be scaled or deployed independently."*

Backend architecture (this is where you impress them):

*"The backend followed a strict **layered MVC architecture**. At the top, we had **REST controllers** — these were thin. Their only job was to accept the HTTP request, call the appropriate service method, and return a properly structured response with the right HTTP status code. For example, a successful create returns a 201, a validation failure returns a 400, a missing record returns a 404, and any unexpected error returns a 500. Every controller method was wrapped in try-catch blocks so the API always returned a meaningful JSON error message — never a raw stack trace."*

*"Below that, we had the **service layer**, which held all the business logic. This is where validation, data sanitization, and business rules lived. We used **inheritance** here — there was an abstract base service that defined a generic validation contract, and each entity-specific service extended it and overrode the validation method with its own rules. We also had a **utility helper class** with static methods for common operations like formatting names, validating emails, and checking for blank inputs — so we weren't repeating that logic everywhere."*

*"The service layer also integrated with **Redis for caching**. Read-heavy endpoints cached their results, and any create, update, or delete operation would evict the cache to keep data fresh. This significantly reduced database load."*

*"At the bottom was the **data access layer**. We used Spring's JdbcTemplate with parameterized SQL queries to prevent SQL injection. Each DAO had a row mapper that converted database result sets into Java objects. The data access layer returned raw data — it didn't throw business exceptions. Instead, it returned null or row-affected counts, and the service layer interpreted those results and threw appropriate exceptions."*

Authentication (show you understand security):

*"Authentication was handled by a completely separate microservice. When a user logged in, the auth service verified their credentials against a BCrypt-hashed password stored in the database, and issued a **JWT token** signed with HMAC-SHA256. The token contained the username, role, and an expiry timestamp."*

"The core backend didn't know how to validate JWTs at all. We implemented it as a **request interceptor** — every incoming API request was intercepted, and the interceptor made a synchronous call to the auth service's validation endpoint. If the token was valid, the request proceeded. If not, the user got a 401. This kept authentication logic fully decoupled from business logic."

Frontend (keep it brief):

"The frontend was a React SPA. We used a context provider for auth state management, which stored the JWT token in localStorage. An axios interceptor automatically attached the token to every outgoing request. If any response came back as 401, another interceptor would clear the stored credentials and redirect to the login page. All routes except login were protected — if you weren't authenticated, you'd get redirected."

CORS (mention if they ask about cross-origin):

"Since the frontend and backend ran on different ports, we configured CORS on the backend to allow requests from the frontend's origin. We whitelisted specific HTTP methods, allowed the Authorization header explicitly, and enabled credentials."

Possible Follow-up Questions and How to Answer

Question	Your Answer
"Why a separate auth service?"	"Separation of concerns. If we later added another backend — say an analytics service — it could reuse the same auth service without duplicating any security logic."
"Why JWT instead of sessions?"	"Stateless scalability. No server-side session store needed. Any instance of the auth service can validate any token because they all share the same signing key. JWT fits microservices better than sessions."
"What database did you use?"	"Oracle. We used separate schemas for each service — the auth service had its own schema, the CRUD backend had its own. This way one service's traffic doesn't affect the other."
"How did you handle caching?"	"Redis. We cached read-heavy queries with a TTL and evicted cache entries on any write operation. The model objects implemented Serializable so Redis could store and retrieve them."
"What if the token is expired?"	"The auth service checks the expiry claim when validating. If expired, it returns unauthorized. The frontend's response interceptor catches the 401, clears stored credentials, and forces a re-login."
"Any drawback of calling auth service on every request?"	"Yes, it's chatty. In production I'd optimize it by either caching validation results in Redis with a short TTL, sharing the signing key so the backend validates locally, or putting an API gateway in front that handles auth once before routing."